

Custom Wake-Word Detector

Project Architecture & Technical Report

A from-scratch machine learning system that detects a personal spoken wake word ("Hello Nova") using a custom-trained Convolutional Neural Network on MFCC audio features.

Table of Contents

Project Architecture & Technical Report	1
Table of Contents	2
1. Project Overview	3
1.1 Core Concepts Involved	3
2. System Architecture Overview	4
2.1 Folder Structure	4
3. Data Collection	5
3.1 Positive Samples	5
3.2 Negative Samples	5
3.3 Class Imbalance	5
3.4 Recording & Conversion Pipeline Used	5
4. Feature Extraction (MFCCs)	6
4.1 The Extraction Pipeline	6
4.2 Implementation Details	6
5. Model Architecture (CNN)	7
5.1 Why a CNN	7
5.2 Why Small, Specifically	7
5.3 Architecture Breakdown	7
6. Training & Evaluation	8
6.1 Data Splitting	8
6.2 The Training Loop	8
6.3 Evaluation Metrics — Why Accuracy Alone Is Misleading	8
7. Development & Iteration History	9
8. Current Project Status	10
9. Planned Next Steps	11
9.1 Immediate: Expand Positive Dataset	11
9.2 Stage 5: Real-Time Detection Loop	11
9.3 The Iteration Loop	11
9.4 Potential Extensions	11
10. Concept Glossary	12

1. Project Overview

This project implements a personal wake-word detector: a system that continuously listens through a microphone and recognizes a single custom spoken phrase, similar in concept to "Hey Siri" or "Alexa", but built and trained entirely from scratch on the developer's own voice data rather than relying on a pre-trained third-party API.

Unlike projects built on calling an existing AI service, every part of the detection logic here — data collection, feature extraction, model design, training, and evaluation — is implemented and trained locally.

1.1 Core Concepts Involved

Concept	Where It Is Used
Dataset design & class balance	Data collection stage
Digital signal processing (Fourier Transform, Mel scale)	Feature extraction stage
Convolutional Neural Networks (CNNs)	Model architecture
Gradient descent & backpropagation	Training loop
Train / validation / test splitting, overfitting	Training & evaluation
Precision, recall, F1, false positive/negative tradeoffs	Evaluation
Real-time streaming & sliding windows	Live detection stage (planned)

2. System Architecture Overview

The project is organized as a linear pipeline, where each stage's output becomes the next stage's input. The diagram below (described textually) represents the flow:

- Raw voice recordings (WhatsApp voice notes, .ogg format)
- → Format conversion (ffmpeg): .ogg → .wav, 16kHz mono
- → Organized dataset: data/positive/ and data/negative/
- → Feature extraction (extract_features.py): .wav → MFCC arrays → features.npz
- → Model training (train.py + model.py): features.npz → wake_word_model.pt
- → Inference (predict.py): new audio file → trained model → prediction + confidence
- → (Planned) Real-time detection loop: live microphone stream → continuous prediction

2.1 Folder Structure

```
voice_recognizer/ |— raw_voice_notes/      Temporary holding area for exported voice
notes |— data/ |   |— positive/             "Hello Nova" recordings, labeled 1 |   |—
negative/         Everything else, labeled 0 |— model.py                 CNN
architecture definition |— extract_features.py  Converts .wav files into MFCC
features |— train.py                      Trains the CNN on extracted features |— predict.py
Runs the trained model on a single new file |— features.npz              Generated:
extracted feature dataset |— wake_word_model.pt      Generated: trained model
weights
```

3. Data Collection

A personal wake-word detector requires the developer to generate their own training data, since no public dataset contains recordings of a custom, self-chosen phrase said in the developer's own voice.

3.1 Positive Samples

Recordings of the phrase "Hello Nova" spoken by the developer. Diversity is intentionally introduced across several dimensions to help the model generalize:

- Distance from microphone: close, arm's length, across the room
- Tone: normal, quiet, excited, tired, rushed
- Speed: normal, faster, slower
- Recording sessions spread across multiple days, to capture natural day-to-day vocal variation

3.2 Negative Samples

Recordings of everything that should NOT trigger detection: silence, background noise, unrelated speech, and — most importantly — phonetically similar phrases (e.g. "Hello Nora"), which teach the model a precise decision boundary rather than a vague one.

3.3 Class Imbalance

Negative examples are deliberately kept in the majority (a ratio of roughly 3:1 or higher against positives), because in real-world use the system will hear far more non-wake-word audio than wake-word audio. Training on an artificially balanced 50/50 split would teach the model an unrealistic expectation of how often the wake word actually occurs, biasing it toward false triggers.

3.4 Recording & Conversion Pipeline Used

Voice notes were recorded via WhatsApp (Opus codec, .ogg container) and exported to the local machine. Since the feature extraction code requires standard .wav audio at 16kHz mono, each file is converted using ffmpeg:

```
ffmpeg -i input.ogg -ar 16000 -ac 1 output.wav
```

Converted files are then copied into data/positive/ or data/negative/ with sequential filenames (e.g. wake_001.wav, neg_001.wav).

4. Feature Extraction (MFCCs)

Raw audio is a waveform: tens of thousands of amplitude values per second. This is far too large and unstructured to feed directly into a neural network, so it is converted into Mel-Frequency Cepstral Coefficients (MFCCs) — a compact numerical representation modeled on how human hearing perceives sound.

4.1 The Extraction Pipeline

Step	What It Does
1. Windowing	Audio is split into short overlapping ~25ms windows, since speech content changes too quickly to analyze as one block.
2. Fourier Transform	Converts each window from amplitude-over-time into a breakdown of which frequencies are present and how strong each is.
3. Mel Scale Mapping	Re-maps frequency values to match human pitch perception, which is more sensitive to differences at lower frequencies.
4. Cepstral Compression	Compresses the Mel-scaled spectrum into a small number of coefficients (13 in this project) per time window.

The result for each audio file is a 2D grid of shape (13 coefficients $\times$ time windows) — structurally similar to a small grayscale image, which is why a Convolutional Neural Network (normally used for image recognition) is a natural fit for this data.

4.2 Implementation Details

Implemented in `extract_features.py` using the `librosa` library:

```
def extract_mfcc(filepath, n_mfcc=13, sample_rate=16000):    audio, sr =  
librosa.load(filepath, sr=sample_rate)    mfccs = librosa.feature.mfcc(y=audio, sr=sr,  
n_mfcc=n_mfcc)    return mfccs # shape: (13, time_windows)
```

Because different recordings have slightly different durations, the resulting MFCC grids have different widths. Since every sample must have an identical shape to be processed in batches, each grid is padded with zeros or trimmed to a fixed 63 time-frames (corresponding to roughly 2 seconds of audio):

```
if mfccs.shape[1] < FIXED_FRAMES:    mfccs = np.pad(mfccs, ((0,0),(0, FIXED_FRAMES -  
mfccs.shape[1]))) else:    mfccs = mfccs[:, :FIXED_FRAMES]
```

All processed samples and their labels (1 = positive, 0 = negative) are bundled and saved into a single `features.npz` file, avoiding the need to re-extract features every time the model is retrained.

5. Model Architecture (CNN)

Since each MFCC grid resembles a small image, a compact Convolutional Neural Network is used, implemented in `model.py` using PyTorch.

5.1 Why a CNN

A CNN learns small, reusable pattern detectors (filters) that slide across the input, detecting local structure. In this project's case, that means detecting characteristic short-duration frequency patterns rather than image edges. Stacking multiple convolutional layers allows the network to build from simple local patterns up to more complex, whole-phrase recognition.

5.2 Why Small, Specifically

Since this model must eventually run continuously in real time, model size directly affects whether it can keep up with incoming audio. A larger model might be marginally more accurate but too slow for real-time use; a small, well-designed model runs instantly with only a modest accuracy cost.

5.3 Architecture Breakdown

Layer	Purpose
Conv2D (1 → 8 channels, 3×3 kernel)	Detects simple local frequency patterns across the MFCC grid.
MaxPool2D (2×2)	Shrinks the grid by half, retaining the strongest signal in each region.
Conv2D (8 → 16 channels, 3×3 kernel)	Combines simple patterns from layer 1 into more complex, phrase-level patterns.
MaxPool2D (2×2)	Further shrinks the grid.
Fully Connected (→ 32 units)	Combines all spatial features into a compact representation.
Fully Connected (→ 2 units)	Final output: one score for "not wake word", one for "wake word".

The input/output size feeding into the first fully-connected layer is computed dynamically from the MFCC grid dimensions, avoiding a common class of shape-mismatch bugs when the audio length or coefficient count changes.

6. Training & Evaluation

6.1 Data Splitting

The dataset is split into three parts using scikit-learn's `train_test_split`, with stratification to preserve the positive/negative ratio in every split:

Split	Proportion	Purpose
Train	70%	What the model directly learns from
Validation	15%	Monitored during training to check progress without contaminating final evaluation
Test	15%	Held out completely, checked only at the end for an honest performance measure

6.2 The Training Loop

Implemented with PyTorch, using the Adam optimizer and Cross-Entropy Loss, run for 30 epochs (full passes over the training data). Each epoch performs the standard gradient descent cycle:

- Forward pass: the model makes predictions on the training batch
- Loss calculation: `CrossEntropyLoss` measures how wrong those predictions were
- Backward pass: `loss.backward()` computes the gradient — the direction each internal weight should shift to reduce error
- Optimizer step: `optimizer.step()` applies that shift to the model's weights

Both training loss and validation loss are printed each epoch, allowing early detection of overfitting (train loss decreasing while validation loss rises).

6.3 Evaluation Metrics — Why Accuracy Alone Is Misleading

Because most audio the system will hear in practice is NOT the wake word, a model that simply never triggers could show high raw accuracy while being completely useless. Precision and recall separate two different kinds of correctness:

Metric	Meaning	Failure Mode If Low
Precision	Of all times the model said "wake word detected", how often was it right?	Low precision = frequent false alarms
Recall	Of all times the wake word was actually said, how often did the model catch it?	Low recall = the system frequently ignores the user

7. Development & Iteration History

Machine learning projects rarely work correctly on the first attempt. The table below documents the actual iteration history of this project, since it illustrates the debugging process as much as the final result.

Iteration	Dataset	Result	Diagnosis
1	6 positive / 18 negative (24 total)	Precision/Recall/F1 all 0.00; loss decreased normally	Test split too small (~3-4 samples) to produce meaningful metrics — a dataset-size artifact, not a broken pipeline.
2	36 positive / 18 negative (54 total)	Precision 0.62, Recall 0.83, F1 0.71	Positive-skewed ratio caused a "trigger-happy" model: good at catching the wake word, prone to false alarms.
3	36 positive / 120 negative (156 total, 100 synthetic TTS negatives added)	Identical confidence (85.9%) across multiple distinct test files	Model collapsed: synthetic TTS negatives were too acoustically uniform, causing the model to learn a crude "robotic vs. human voice" shortcut instead of genuine wake-word discrimination.
4	36 positive / 60 negative (96 total, TTS negatives trimmed to 40)	Precision 1.00, Recall 0.67, F1 0.80	Collapse fixed; model became overly cautious — confident when correct, but missed a third of genuine wake-word instances due to insufficient positive-side data.

This history reflects the project's central lesson: model architecture and code were correct from an early stage; nearly every quality issue traced back to dataset composition — either class imbalance direction or lack of acoustic diversity within a class.

8. Current Project Status

Component	Status
Recording & conversion pipeline	Complete and validated
Dataset organization (positive/negative folders)	Complete
Feature extraction (extract_features.py)	Complete and validated
Model architecture (model.py)	Complete
Training loop (train.py)	Complete and validated
Single-file inference (predict.py)	Complete and validated
Sufficient positive data volume for reliable recall	In progress — currently the main bottleneck
Real-time microphone detection loop	Not yet built
Live testing & iteration on real-time system	Not started

At present, the entire software pipeline is functional end-to-end: audio in, trained model out, predictions on new files. The primary remaining gap is data volume on the positive side — the current model correctly rejects non-wake-word audio (precision 1.00) but under-recognizes genuine instances of the wake word spoken under new conditions (recall 0.67), indicating the model has not yet seen enough variety of the user's own voice to generalize confidently.

9. Planned Next Steps

9.1 Immediate: Expand Positive Dataset

Increase positive samples from 36 toward a target of 80-100, spread across multiple recording sessions and varying distance, tone, and speed, to improve recall without sacrificing the precision already achieved.

9.2 Stage 5: Real-Time Detection Loop

Implement continuous microphone listening using overlapping sliding windows (e.g. a 1.5 second rolling window, re-checked every 0.5 seconds) so that a wake word spoken across a chunk boundary is not missed. Each window will be converted to MFCC features and passed through the trained model in real time.

9.3 The Iteration Loop

Once live, the system will be tested under realistic conditions (not just quiet, controlled recording setups). Specific failure patterns — such as only working close to the microphone, or confusing a similar-sounding word — will be diagnosed and addressed with targeted new recordings, followed by retraining. This loop is expected to repeat multiple times before the system is considered reliable.

9.4 Potential Extensions

- Multi-word detection: extending to recognize multiple distinct wake phrases
- Speaker-specific activation: combining with speaker verification so only the developer's voice triggers detection
- Noise robustness testing: deliberate testing against background noise (music, traffic, other speakers)
- On-device optimization: model quantization for deployment on constrained hardware (e.g. Raspberry Pi)

10. Concept Glossary

Term	Definition
Gradient Descent	The core training algorithm: repeatedly measuring prediction error and adjusting model parameters in the direction that reduces it.
Loss Function	A single number quantifying how wrong the model's current predictions are; the value gradient descent works to minimize.
Overfitting	When a model learns training data too specifically, including its noise, and fails to generalize to new, unseen data.
Fourier Transform	Converts a signal from time-domain (amplitude over time) to frequency-domain (which frequencies are present and how strongly).
Mel Scale	A frequency scale adjusted to match human pitch perception, emphasizing differences at lower frequencies.
MFCC	Mel-Frequency Cepstral Coefficients — a compact numerical representation of audio derived via windowing, Fourier Transform, and Mel-scale mapping.
Convolutional Neural Network (CNN)	A neural network architecture using sliding filters to detect local patterns in grid-structured data.
Precision	Of all positive predictions made, the proportion that were actually correct.
Recall	Of all actual positive cases, the proportion the model correctly identified.
Class Imbalance	When one category in a dataset vastly outnumbers another, requiring deliberate handling to avoid misleading accuracy.
Windowing / Segmentation	Dividing a continuous signal into analyzable chunks; overlapping windows prevent missing events spanning a chunk boundary.